

Informe del Compilador HULK

David Sánchez

June 23, 2025

Abstract

Este documento presenta un informe técnico sobre el diseño e implementación de un compilador para el lenguaje de programación HULK. Se detallan las fases clave del proceso de compilación, incluyendo el análisis léxico, sintáctico y semántico, así como la generación de código intermedio y su ejecución mediante un intérprete JIT (Just-In-Time) basado en LLVM. El objetivo principal es proporcionar una visión clara de la arquitectura del compilador, las decisiones de diseño tomadas y las herramientas utilizadas en su desarrollo.

Contents

1 Introducción

1.1 Motivación

El desarrollo de un compilador es un ejercicio fundamental en las ciencias de la computación que permite comprender a fondo la estructura de los lenguajes de programación y el proceso de traducción de código fuente a código ejecutable. Este proyecto fue motivado por el deseo de aplicar los conceptos teóricos de la compilación en un proyecto práctico y completo, abordando los desafíos de cada una de sus fases.

1.2 El Lenguaje HULK

HULK es un lenguaje de programación diseñado para este proyecto, que incluye características como tipos definidos por el usuario, funciones, herencia y un sistema de tipos estático. Su sintaxis y semántica están pensadas para ser lo suficientemente ricas como para explorar los problemas clásicos de la compilación.

1.3 Objetivos

Los objetivos principales de este proyecto son:

- Diseñar e implementar un analizador léxico y sintáctico para el lenguaje HULK.
- Construir un Árbol de Sintaxis Abstracta (AST) como representación intermedia del código fuente.
- Implementar un analizador semántico robusto para la validación de tipos y reglas de alcance.
- Generar código intermedio utilizando la infraestructura de LLVM.
- Integrar un intérprete JIT para la ejecución dinámica del código generado.

1.4 Estructura del Informe

Este informe está organizado de la siguiente manera: las secciones 2, 3 y 4 describen los análisis léxico, sintáctico y semántico, respectivamente. La sección 5 detalla el proceso de generación de código. Finalmente, se presentan las conclusiones y el trabajo futuro.

2 Análisis Léxico

2.1 Tokens

El analizador léxico se encarga de leer el código fuente y agruparlo en una secuencia de tokens. Los tokens representan las unidades mínimas con significado del lenguaje, como palabras clave ('if', 'else', 'while'), identificadores, números, cadenas y operadores.

2.2 Expresiones Regulares

Cada tipo de token se define mediante una expresión regular que describe el patrón de caracteres que lo conforma. Por ejemplo, un identificador podría ser `[a-zA-Z][a-zA-Z0-9_]*`.

2.3 Implementación

El analizador léxico se implementó utilizando la herramienta Flex, un generador rápido de analizadores léxicos. A partir de un archivo de especificación ('lexer.l') que contiene las expresiones regulares y las acciones asociadas, Flex genera el código C++ del analizador.

3 Análisis Sintáctico

3.1 Gramática

La estructura sintáctica del lenguaje HULK se define mediante una gramática libre de contexto, presentada en formato BNF (Backus-Naur Form). Esta gramática especifica cómo se pueden combinar los tokens para formar construcciones válidas como expresiones, declaraciones y sentencias. La gramática completa se encuentra en el apéndice de este informe.

3.2 Tipo de Parser

Se utilizó un parser de tipo LALR(1) (Look-Ahead LR), que es capaz de analizar un amplio rango de gramáticas de forma eficiente.

3.3 Árbol de Sintaxis Abstracta (AST)

El resultado del análisis sintáctico es un Árbol de Sintaxis Abstracta (AST). El AST es una representación jerárquica del código fuente que captura su estructura esencial, eliminando detalles sintácticos irrelevantes como paréntesis o puntos y comas. Este árbol es la estructura de datos principal que se utiliza en las fases posteriores del compilador.

3.4 Implementación

El analizador sintáctico se implementó con la herramienta Bison, un generador de parsers compatible con Yacc. A partir de un archivo de gramática ('parser.y'), Bison genera el código C++ del parser que, en conjunto con el analizador léxico, construye el AST.

4 Análisis Semántico

El chequeo semántico se realiza en tres fases principales, cada una implementada por un visitante (visitor) especializado que recorre el AST. Se utiliza un contexto global para almacenar información sobre tipos, variables y funciones.

4.1 Type Collector

Este visitor se encarga de recolectar las definiciones de tipos en el programa.

- **Lógica:** Recorre el AST y registra los tipos definidos por el usuario (clases, protocolos, etc.) en el contexto global.
- **Propósito:** Garantizar que todos los tipos estén definidos antes de ser utilizados.

4.2 Symbol Collector

Este visitor recolecta las definiciones de variables y funciones.

- **Lógica:** Recorre el AST y registra las variables, funciones y métodos en el contexto correspondiente.
- **Propósito:** Construir las tablas de símbolos para cada ámbito (global, local, de clase) y verificar que no haya redefiniciones.

4.3 Semantic Checker

Este visitor realiza el análisis semántico detallado.

- **Lógica:** Verifica que las operaciones sean válidas según los tipos de datos, que las variables estén declaradas antes de ser usadas y que las funciones sean llamadas con los argumentos correctos.
- **Propósito:** Detectar errores semánticos como tipos incompatibles, uso de variables no declaradas o violaciones de las reglas del lenguaje.

4.4 Contexto y Búsqueda Recursiva

El contexto es una estructura central que almacena la información semántica. Cuando se busca una variable, función o tipo, el contexto realiza una búsqueda recursiva desde el ámbito actual hacia los ámbitos superiores, manejando correctamente los ámbitos anidados y la herencia.

5 Generación de Código

El generador de código está basado en un patrón visitor que recorre el AST y traduce cada nodo a instrucciones de bajo nivel utilizando la infraestructura de LLVM.

5.1 Representación Intermedia (LLVM IR)

Se utiliza LLVM Intermediate Representation (IR) como código intermedio. LLVM IR es un lenguaje de ensamblador de bajo nivel con un formato de tres direcciones (Three-Address Code), lo que permite que el código generado sea portable y susceptible a múltiples optimizaciones.

5.2 Visitor de Generación de Código

El visitor de generación de código implementa métodos para cada tipo de nodo del AST. Su función es transformar las construcciones del lenguaje HULK en instrucciones de LLVM IR.

- Gestiona la correspondencia entre variables del código fuente y sus representaciones en LLVM.
- Genera la declaración de funciones, la reserva de memoria para variables y los bloques básicos para estructuras de control (condicionales, bucles).

5.3 Intérprete JIT (Just-In-Time)

Una vez generado el código LLVM IR, el compilador utiliza el intérprete JIT de LLVM para ejecutar el programa directamente en memoria, sin necesidad de compilar a un binario nativo. El JIT traduce dinámicamente el IR a código máquina para la plataforma actual y lo ejecuta, facilitando las pruebas y el desarrollo interactivo.

6 Pruebas

7 Conclusiones

7.1 Resumen del Trabajo

Este proyecto ha culminado con la implementación de un compilador funcional para el lenguaje HULK. Se ha desarrollado una arquitectura modular basada en visitors que facilita la separación de responsabilidades entre las distintas fases del compilador. El uso de herramientas estándar como Flex y Bison, junto con la potente infraestructura de LLVM, ha permitido construir un compilador robusto y extensible.

7.2 Trabajo Futuro

Como trabajo futuro, se podrían explorar varias mejoras:

- Implementar un sistema de módulos para organizar mejor el código.
- Añadir más optimizaciones al código generado por LLVM.
- Mejorar el reporte de errores con información más detallada sobre la ubicación y causa de los mismos.
- Extender el lenguaje con nuevas características como genéricos o programación concurrente.

References

- [1] Aho, A. V., Lam, M. S., Sethi, R., Ullman, J. D. (2007). *Compilers: Principles, Techniques, and Tools*. Pearson/Addison Wesley.

A Gramática Completa

```
input -> decl_list lines_block
      | lines_block
      | line lines_block

decl_list -> epsilon
          | declarations

declarations -> decl
             | declarations decl

decl -> func_full_assign
      | type_node_decl

lines_block -> { lines }

lines -> epsilon
       | non_empty_lines

non_empty_lines -> line
                 | non_empty_lines line

line -> expr ;
      | func_def ;

expr -> arit_op
      | bool_expr
      | while while_expr
      | string
      | id_expr
      | func_call
      | let_exp
      | if conditional
      | member_access_expr
      | expr := expr
      | expr = expr
      | expr @ expr
      | expr @@ expr

func_def -> function var_id ( args_list ) => expr
          | function var_id ( args_list ) : type_id => expr

func_full_assign -> function var_id ( args_list ) { lines }
                  | function var_id ( args_list ) : type_id { lines }

args_list -> epsilon
           | id_expr
           | args_list , id_expr

id_expr -> var_id : type_id
         | var_id

let_exp -> let var_assign_list in expr
         | let var_assign_list in lines_block

var_assign_list -> id_expr = expr
                 | id_expr = expr as type_id
```

```

    | id_expr = new_expr
    | var_assign_list , id_expr = expr
    | var_assign_list , id_expr = expr as type_id
    | var_assign_list , id_expr = new_expr

new_expr -> new var_id ( expr_list )

expr_list -> epsilon
    | expr
    | new_expr
    | expr_list , expr
    | expr_list , new_expr

func_call -> var_id ( expr_list )

arit_op -> arit_op + term
    | arit_op - term
    | term

term -> term * factor
    | term / factor
    | term % factor
    | factor

factor -> sign ^ factor
    | sign

sign -> atom
    | - atom

atom -> number
    | var_id
    | ( expr )
    | func_call
    | member_access_expr

bool_expr -> bool
    | expr >= expr
    | expr > expr
    | expr <= expr
    | expr < expr
    | expr == expr
    | expr != expr
    | expr & expr
    | expr | expr
    | ! expr
    | id_expr is type_id
    | func_call is type_id

conditional -> ( bool_expr ) expr else expr
    | ( bool_expr ) lines_block else expr
    | ( bool_expr ) expr else lines_block
    | ( bool_expr ) lines_block else lines_block
    | ( bool_expr ) expr elif conditional
    | ( bool_expr ) lines_block elif conditional

while_expr -> ( bool_expr ) lines_block
    | ( bool_expr ) expr

```

```

type_node_decl -> type type_id { type_body_elements }
| type type_id inherits type_id { type_body_elements }
| type type_id inherits type_id ( args_list ) { type_body_elements }
| type type_id ( args_list ) { type_body_elements }
| type type_id ( args_list ) inherits type_id { type_body_elements }
| type type_id ( args_list ) inherits type_id ( args_list ) { type_body_elements }

type_body_elements -> epsilon
| type_body_elements attribute
| type_body_elements method

attribute -> id_expr = expr ;
| id_expr = expr as type_id ;

method -> var_id ( args_list ) => expr ;
| var_id ( args_list ) : type_id => expr ;
| var_id ( args_list ) { lines }
| var_id ( args_list ) : type_id { lines }

member_access_expr -> expr . var_id
| expr . func_call

```

B Fragmentos de Código Fuente

Listing 1: Ejemplo de código C++

```
// Tu código aquí
```